



DOCKER CHEAT SHEET

Compact v2 · Engine 23+/27+ · Compose v2 · Buildx · Swarm · full detail: [docker-cheat-sheet.md](#) · hover any block → copy · row-first layout

Containers Images Registry Networks Volumes System Build Compose Swarm Dockerfile Reference Careful

QUICK START

01

```
# verify the installation
docker run hello-world

# detached web server
docker run -d -p 8080:80 --name web nginx

# running containers
docker ps

# follow logs
docker logs -f web

# shell into the container
docker exec -it web bash

# stop and remove
docker stop web
docker rm web

# clean everything unused
docker system prune
```

CLI STRUCTURE

02

docker ps	= docker container ls
docker images	= docker image ls
docker rmi	= docker image rm
docker build	= buildx build (BuildKit)
pull / push	= image pull / push

Mgmt: container image network volume system context
builder buildx compose swarm node service stack secret
config plugin manifest trust scout checkpoint debug
desktop

```
docker CONTAINER SUBCOMMAND ARG...
```

CONTAINER LIFECYCLE

03

- `run` create+start · `create` only create
- `start -ai` C start+attach
- `stop -t 30` graceful TERM→KILL
- `restart` · `kill -s SIGINT`
- `pause` / `unpause` freeze procs
- `rename OLD NEW`
- `rm -f -v` force + anon volumes
- `update --memory 512m` C live limits
- `wait` C block till exit
- `container prune` **del ALL stopped**

EXEC · ATTACH · COPY

04

```
# shell – debian/ubuntu images
docker exec -it web bash

# shell – alpine images
docker exec -it alpine1 sh

# background, env var, other user
docker exec -d web touch /tmp/x
docker exec -e F=web env
docker exec -u root web whoami

# attach to PID 1 output
docker attach web

# detach WITHOUT stopping: Ctrl+p Ctrl+q
# custom detach keys
docker attach --detach-keys="ctrl-a,a" web

# copy out / in
docker cp web:/etc/nginx/nginx.conf ./
docker cp ./app.js web:/srv/app.js

# flattened filesystem tar
docker export web -o fs.tar
cat fs.tar | docker import - img:1

# snapshot container as image
docker commit -m msg -a me C img:v2
```

RUN — FLAGS BY GROUP

05

CORE

- `-d` bg · `-it` tty · `--rm` auto-del · `--name`
- `--entrypoint sh` · `--restart` unless-stopped
- `--platform linux/arm64` · `--pull` always · `--init` tini PID1
- `--link C2:alias` **legacy ⚠** — use networks instead

PORTS

- `-p 8080:80` pub · `-p 127.0.0.1:80:80` loopback
- `-P` publish all EXPOSED→random host ports

ENV & CONFIG

- `-e K=v` · `--env-file .env` · `-w /dir` cwd
- `-u user` · `-h host` · `--label k=v` · `--log-driver json-file`
- `--dns 8.8.8.8` · `--add-host db:ip` · `--sysctl k=v` · `--ulimit n=s:h`

STORAGE

- `-v vol:/data` named volume
- `-v $(pwd)/src:/dst[:ro]` bind mount
- `--tmpfs /tmp` ramdisk
- `--mount type=bind,src=.,tgt=.,[readonly]`
- `type=volume,src=vol,tgt=/d` · `type=tmpfs,dst=/t,size=100m`
- `--volumes-from C2` reuse mounts

RESOURCES

- `-m 512m` · `--memory-swap 1g` · `--cpus 1.5`
- `--cpu-shares 512` · `--cpuset-cpus 0-3` · `--pids-limit 100`

SECURITY

- `--cap-add SYS_ADMIN` / `--cap-drop ALL` · `--device /dev/x`
- `--read-only` rootfs · `--security-opt no-new-privileges`
- `--privileged` **⚠ avoid — full host access**

NETWORKING

- `--network mynet` bridge/host/none/custom

RESTART POLICIES

06

no	never (default)
on-failure[:N]	non-zero exit, max N
always	incl. daemon restart
unless-stopped	stays stopped if manual stop

```
# change policy on a live container
docker update --restart=always C
```

INFO & DEBUG

07

- `ps -a -q -l -s -n5` all/ids/latest/sizes
 - `logs -f --tail 100 -t --since 10m`
 - `inspect -s` · `-f '{{.State.Status}}'`
 - `top` · `stats` · `--no-stream -a`
 - `port C` · `diff C` A/C/D files
 - `events --filter event=start`
- ```
healthcheck states: starting → healthy/unhealthy
docker inspect -f '{{.State.Health.Status}}' web

define on run
docker run -d --health-cmd "curl -f localhost" \
 --health-interval 30s --health-timeout 5s \
 --health-retries 3 --health-start-period 10s nginx

docker events --filter event=health_status
```

## IMAGES

08

- `images -a -q --digests --tree` (tree: 27+)
- `pull img:tag` · `--platform linux/amd64` · `-a`
- `push` · `tag SRC DST` · `rmi -f IMG`
- `history --no-trunc IMG` layer audit
- `image inspect` · `image prune -a`
- `search term --filter stars=50 --limit 25`
- `save -o t.tar img:1` ↔ `load -i t.tar` keeps history
- `save img | gzip > t.gz` ↔ `gzip -dc t.gz | docker load`
- export/import = flat 1-layer, history lost
- pin versions: `nginx:1.27-alpine` ≠ `:latest`
- immutable pin: `img@sha256:...`

## REGISTRIES

09

```
docker login
docker login -u user registry.example.com

CI-safe login (never -p PASSWORD)
echo $TOKEN | docker login ghcr.io \
-u user --password-stdin
docker logout registry.example.com

fully-qualified name hits that registry
docker pull registry.example.com/team/app:1.2

run a local private registry
docker run -d -p 5000:5000 --name reg \
--restart always registry:2

publish into it
docker tag app localhost:5000/app:1
docker push localhost:5000/app:1
```

## NETWORKS

010

`bridge` `host` `none` `overlay` `macvlan` — custom bridge ⇒ DNS by container name

```
create (bridge is default driver)
docker network create mynet
docker network create -d overlay net
no external egress
docker network create --internal airgapped
docker network create \
--subnet=172.20.0.0/16 \
--gateway=172.20.0.1 mynet

inspect / list
docker network ls
docker network inspect mynet

attach / detach running container
docker network connect mynet web
docker network disconnect mynet web

docker network rm mynet
docker network prune

static IP inside custom subnet
docker run -d --net mynet \
--ip 203.0.113.2 nginx
```

## VOLUMES

011

`named -v vol:/d` · `bind -v /h:/c` · `tmpfs --tmpfs /t`

```
docker volume create mydata
docker volume ls -qf dangling=true
docker volume inspect mydata
docker volume rm mydata
docker volume prune -a

use a named volume
docker run -v mydata:/var/lib/pg/data pg:16

reuse another container's volumes
docker run --volumes-from datastore busybox

backup a volume
docker run --rm -v mydata:/d -v $(pwd):/b \
alpine tar czf /b/v.tgz -C /d .

restore a volume
docker run --rm -v mydata:/d -v $(pwd):/b \
alpine tar xzf /b/v.tgz -C /d

single-file config mount (read-only)
docker run -v $(pwd)/app.conf:/etc/app.conf:ro httpd
```

## SYSTEM & CLEANUP

012

```
docker version
docker info
docker system df -v
docker system events
```

| prune...                                  | removes                |
|-------------------------------------------|------------------------|
| <code>container prune</code>              | stopped ctors          |
| <code>image prune [-a]</code>             | dangling [unused]      |
| <code>network prune</code>                | unused nets            |
| <code>volume prune [-a]</code>            | unused volumes         |
| <code>builder prune -a</code>             | build cache            |
| <code>system prune</code>                 | above – volumes        |
| <code>system prune -a --volumes -f</code> | <b>nuke all unused</b> |

filters: `--filter until=24h` `--filter label=k=v`

## CONTEXTS

013

```
docker context ls
docker context show

remote daemon over SSH
docker context create prod \
--docker host=ssh://user@srv

plain TCP endpoint
docker context create tcpctx \
--docker host=tcp://1.2.3.4:2375

docker context use prod

per-invocation override
DOCKER_CONTEXT=prod docker ps

docker context inspect prod
docker context rm prod
```

## BUILD & BUILDX

014

```
docker build -t app:1 .

custom file / args / cache control
docker build -f Dockerfile.prod \
--build-arg V=1 --no-cache --pull \
--target stage -o out .

BuildKit secret (never in layers)
DOCKER_BUILDKIT=1 docker build \
--secret id=tok,src=/token.txt -t app .

docker buildx ls
docker buildx create --use --name b1

multi-arch build + push
docker buildx build \
--platform linux/amd64,linux/arm64 \
-t u/app:1 --push .

load result into local engine
docker buildx build --load -t app .

docker buildx du
docker buildx prune -f
docker buildx imagetools inspect u/app:1
```

## COMPOSE — CONTROL

015

- `up -d --build --wait --force-recreate --no-deps`
- `up -d --scale worker=4`
- `down -v --rmi all --remove-orphans`
- `stop` / `start` / `restart svc`
- `pause` / `unpause` · `kill -s SIGINT`
- `rm -sf svc` · `create` · `ls` projects

### GLOBAL FLAGS

- `-f f.yaml` repeatable · `-p proj`
- `--project-directory DIR` · `--env-file f`
- `--profile debug up` optional profiles

## COMPOSE — SERVICES

016

- `ps -a --services` · `top` · `port web 80`
- `images` · `events` · `logs -f web db`
- `exec web sh` running svc
- `run --rm web pytest` one-off container
- `config -q` validate · `build --no-cache`
- `pull / push` service images
- `watch` live sync/rebuild, ≥2.22

```
services:
 web:
 build: .
 ports: ["8080:80"]
 env_file: .env
 restart: unless-stopped
 depends_on:
 cache:
 condition: service_healthy
 healthcheck:
 test: ["CMD", "wget", "-q0-", "http://localhost/"]
 interval: 30s
 retries: 3
 develop:
 watch:
 - action: sync
 path: ./src
 target: /app/src
 - action: rebuild
 path: ./package.json
 cache:
 image: redis:7-alpine
 volumes: {}
```

## SWARM MODE

017

```
init manager / join tokens
docker swarm init --advertise-addr 10.0.0.1
docker swarm join-token worker
docker swarm join-token manager

on a worker node
docker swarm join --token SWMTKN-... 10.0.0.1:2377
docker swarm leave
docker swarm leave --force # manager must force

nodes
docker node ls
docker node promote n2
docker node demote n2
docker node update --availability drain n3
```

### SERVICES — ROLLING DEPLOYS

```
docker service create --name web \
 --replicas 3 -p 80:80 nginx
docker service ls
docker service ps web
docker service logs -f web
docker service inspect --pretty web

rolling image update + rollback
docker service update --image nginx:1.27 web
docker service rollback web
docker service scale web=5
docker service rm web
```

### STACK · SECRET · CONFIG

```
docker stack deploy -c compose.yaml mystack
docker stack ls
docker stack ps mystack
docker stack services mystack
docker stack rm mystack

secrets (mounted at /run/secrets/<name>)
printf "s3cr3t" | docker secret create db_pass -
docker secret ls
docker secret inspect db_pass
docker secret rm db_pass
docker service create --secret db_pass api

configs
docker config create ngx ./nginx.conf
docker service create --config ngx proxy
```

## MISC COMMANDS

018

- `manifest inspect img` multi-arch check
- `scout cves img` CVE scan (replaces scan)
- `scout recommendations img`
- `trust sign/inspect/revoke/key`
- `plugin install/store/lr/rm`
- `checkpoint create/lr/rm` exp · CRIU
- `debug img` toolbox shell
- `desktop start|stop|status` 4.37+

## DOCKERFILE INSTRUCTIONS

019

- `FROM img[:tag] [AS st]` base/stage alias
  - `RUN cmd` layer at build · chain with `&&`
  - `CMD ["e", "a"]` default cmd · last wins
  - `ENTRYPOINT ["e"]` fixed exe · `CMD = args`
  - `ENV K=V` runtime env
  - `ARG K=d` build-only ( `--build-arg` )
  - `COPY src dst [--chown=u:g]` prefer over `ADD`
  - `ADD src dst` `COPY+tar+URL` ⚠
  - `WORKDIR /p` · `USER u` · `LABEL k=v`
  - `EXPOSE 80` docs only — ≠ **publish!**
  - `VOLUME /d` · `SHELL ["bash", "-c"]`
  - `STOPSIGNAL SIGTERM` · `ONBUILD ...`
  - `HEALTHCHECK CMD curl -f http://localhost/ || exit 1`
- Exec form** `["nginx", "-g", "off;"]` receives `SIGTERM`; shell form swallows signals → use exec form or run with `--init`.

## MULTI-STAGE TEMPLATE

020

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup -S app \
 && adduser -S app -G app
COPY --from=deps /app/node_modules \
 ./node_modules
COPY --from=build /app/dist ./dist
USER app
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

## FILTERS & FORMATTING

021

```
ps -f: status=exited ancestor=nginx name=web label=k=v
volume=v network=n publish=8080 expose=443
health=unhealthy exited=137 before/since=id
images -f: dangling=true reference=app:* label=k=v
volume ls -f: dangling=true driver=local
network ls -f: type=custom scope=swarm

custom columns
docker ps --format "{{.ID}} {{.Names}} {{.Ports}}"
docker ps --format "table {{.Names}}\t{{.Image}}"
docker images --format "{{.Repository}}:{{.Tag}} {{.Size}}"

raw JSON of any object
docker inspect -f '{{json .Config.Labels}}' web | jq .

template helpers: json · upper · lower · split · printf

container IP per network
docker inspect -f \
 '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
web
```

## .DOCKERIGNORE

022

```
.git
.gitignore
node_modules
npm-debug.log*
dist
coverage
.env
.env.*
*.md
Dockerfile*
docker-compose*.yaml
tests/
.vscode
.idea
Thumbs.db
.DS_Store
```

## SLIM IMAGES CHECKLIST

023

- **alpine** **-slim** **distroless** smallest base that works
- clean caches in the **same RUN layer**: `apk del`, `rm -rf /var/cache/*`
- deps first, source last — maximize cache hits
- multi-stage: only artifacts reach runtime stage
- `.dockerignore` aggressively — smaller context
- audit bloat: `history --no-trunc img`

## ▶ WORKFLOW RECIPES

024

### DEV LOOP

```
start with rebuild
docker compose up --build -d

live file sync (compose.yaml develop:)
docker compose watch

follow app logs
docker compose logs -f api

debug inside the service
docker compose exec api sh

teardown
docker compose down
```

### TROUBLESHOOT A CONTAINER

```
docker ps -a
docker logs --tail 100 name
docker inspect name
docker stats --no-stream
docker exec -it name sh
docker events --filter name=name
```

### SHIP AN IMAGE

```
docker build -t user/app:1.0 .
docker tag user/app:1.0 user/app:latest
docker login
docker push user/app:1.0
docker push user/app:latest

elsewhere
docker pull user/app:1.0
docker run -d -p 80:80 user/app:1.0
```

### CLEANUP LADDER — MILD TO NUCLEAR

```
docker image prune
docker builder prune -a
docker container prune
docker volume prune -a
docker system prune -af --volumes
```

```
check what is eating disk first
docker system df -v
```

## ▶ ONE-LINERS

025

```
stop / kill all running
docker stop $(docker ps -q)
docker kill $(docker ps -q)

remove containers
docker rm -f $(docker ps -aq)
docker rm $(docker ps -aqf status=exited)

remove images
docker rmi -f $(docker images -qf dangling=true)
docker rmi $(docker images -q) # ALL!

remove dangling volumes
docker volume rm $(docker volume ls -qf dangling=true)

throwaway shells
docker run --rm -it alpine sh
docker run --rm -it -v "$(pwd)":/w -w /w alpine sh

build without a Dockerfile
docker build -t http - <<EOF
FROM alpine
RUN apk --no-cache add http
EOF

latest container helpers
docker ps -l -q
docker logs $(docker ps -lq)
```

## ▶ POWERSHELL NOTES

026

```
$() subexpressions work like bash
docker rm -f $(docker ps -aq)
docker system prune -af --volumes
```

```
xargs replacement
docker ps -aq --filter name=old |
 ForEach-Object { docker rm -f $_ }
```

```
backtick = line continuation
docker run --rm -v "${PWD}:/src" `
 -w //src alpine ls
```

- WSL2 backend recommended for Linux containers
- tray icon switches Linux ↔ Windows ctors
- enable drive sharing in settings for binds
- **exit** leaves exec; Ctrl+p/q only detaches **run -it**

## ▶ GOTCHAS

027

- **EXPOSE ≠ published** — only **-p/-P** publishes
- **ENTRYPOINT vs CMD**: args after img replace CMD
- **ARG vs ENV**: neither is secret-safe
- **default bridge has no DNS** by name
- **latest = default tag**, not newest
- rm ctor deletes writable layer — volumes survive
- attached **run -it**: exit stops, Ctrl+p/q detaches
- disk full → **system df** then prune ladder
- compose dir name = project name → collide; use **-p**
- **docker group ≈ root**; never mount **/var/run/docker.sock**

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>▶ DOCKER HUB028</div> <div><pre># authenticate — use a Personal Access Token, # NOT your account password (esp. CI) docker login -u myuser echo \$PAT   docker login -u myuser --password-stdin  # search the Hub docker search nginx --filter is-official=true docker search alpine --filter stars=50 --limit 25  # namespace = username or organization docker pull myorg/teamapp:1.0 docker push myorg/teamapp:1.0</pre></div> <div>HUB API V2 — NO CLI REQUIRED</div> <div><pre># list tags of any image curl -s "https://hub.docker.com/v2/repositories/library/nginx/tags/? page_size=10" \   jq '.results[].name'</pre></div> <div>PUSH · CACHE · MULTI-ARCH VIA HUB</div> <div><pre># multi-arch straight to Hub docker buildx build \   --platform linux/amd64,linux/arm64 \   -t myorg/app:1.0 --push .  # build cache stored IN Hub (fast CI rebuilds) docker buildx build \   --cache-to type=registry,ref=myorg/app:cache,mode=max \   --cache-from type=registry,ref=myorg/app:cache \   -t myorg/app:1.0 --push .  # multi-arch manifest by hand docker manifest create myorg/app:1.0 \   --amend myorg/app:1.0-amd64 \   --amend myorg/app:1.0-arm64 docker manifest push myorg/app:1.0  # always pull fresh before run/build docker run --pull always myorg/app</pre></div> <div><ul style="list-style-type: none"><li>• <b>PAT:</b> Hub → Account Settings → Personal access tokens — never account password in CI</li><li>• rate limits ~100 pulls/6h/IP anon · 200 authed — check <code>Ratelimit-Remaining</code> header</li><li>• <code>library/*</code> = Official Images · badge = Verified Publisher</li><li>• remote tag/repo delete = Hub UI or API v2 only · <b>docker hub</b> plugin = repos/tags/webhooks from terminal</li><li>• private repos need paid plan — or self-hosted <code>registry:2</code></li></ul></div> | <div>▶ BEST PRACTICES029</div> <div><ol style="list-style-type: none"><li>1. One concern per container — disposable units</li><li>2. Pin versions/digests — never ship <code>:latest</code> to prod</li><li>3. Multi-stage builds + smallest viable base</li><li>4. Order layers cold→hot to leverage cache</li><li>5. Non-root <code>USER</code>; read-only FS where possible</li><li>6. Config via env vars; secrets via secret mounts</li><li>7. <b>HEALTHCHECK</b> + proper signal handling (exec form/ <code>--init</code>)</li><li>8. Named volumes for state — never bake data into images</li><li>9. dockerignore everything unneeded</li><li>10. Scan regularly ( <code>scout cves</code> ), refresh base images</li><li>11. User-defined networks + names, not IPs/ <code>--link</code></li><li>12. Limits next to noisy neighbors: memory/cpus/pids</li></ol></div> | <div>▶ SECURITY CHECKLIST030</div> <div><ul style="list-style-type: none"><li>• [] <code>docker</code> group ≈ root — guard membership</li><li>• [] never expose <code>/var/run/docker.sock</code></li><li>• [] non-root <code>USER</code> in every Dockerfile</li><li>• [] <code>--cap-drop ALL</code>, add back only needed caps</li><li>• [] avoid <code>--privileged</code>; isolate if unavoidable</li><li>• [] <code>--read-only</code> rootfs + writable tmpfs/volumes</li><li>• [] limits: <code>-m</code> <code>--cpus</code> <code>--pids-limit</code></li><li>• [] <code>--security-opt no-new-privileges</code></li><li>• [] pull by digest; scan with <code>scout cves</code></li><li>• [] secrets via mounts — never ENV/ARG layers</li><li>• [] registry auth via <code>--password-stdin</code> only</li><li>• [] keep engine + bases patched</li></ul></div> | <div>▶ OFFICIAL RESOURCES031</div> <div><ul style="list-style-type: none"><li>• <a href="https://docs.docker.com/reference/cli">docs.docker.com/reference/cli</a> — full CLI reference</li><li>• <a href="https://docs.docker.com/compose/compose-file">docs.docker.com/compose/compose-file</a> — compose spec</li><li>• <a href="https://docs.docker.com/build">docs.docker.com/build</a> — buildx &amp; BuildKit</li><li>• <a href="https://docs.docker.com/engine/swarm">docs.docker.com/engine/swarm</a></li><li>• <a href="https://docs.docker.com/scout">docs.docker.com/scout</a></li><li>• <a href="https://docs.docker.com/docker-hub">docs.docker.com/docker-hub</a> — Hub docs &amp; rate limits</li><li>• <a href="https://hub.docker.com">hub.docker.com</a> — image search</li><li>• <a href="https://labs.play-with-docker.com">labs.play-with-docker.com</a> — free sandbox</li><li>• <a href="https://killercoda.com/docker">killercoda.com/docker</a> — interactive labs</li></ul></div> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|